

What is JOIN?

JOIN is used to **combine data from multiple tables**

It works using a **common column** (usually Primary Key ↔ Foreign Key)

Tables for Example

Table 1: departments

```
CREATE TABLE departments (  
    dept_id INTEGER PRIMARY KEY,  
    dept_name TEXT  
);  
INSERT INTO departments VALUES  
(1, 'IT'),  
(2, 'HR'),  
(3, 'Finance'),  
(4, 'Marketing');
```

Table 2: employees

```
CREATE TABLE employees (  
    emp_id INTEGER PRIMARY KEY,  
    name TEXT,  
    dept_id INTEGER  
);  
INSERT INTO employees VALUES  
(1, 'Neeraj', 1),  
(2, 'Amit', 2),  
(3, 'Riya', 3),  
(4, 'Sohan', NULL),  
(5, 'Priya', 5);
```

Note:

- dept_id = 5 does NOT exist in departments
- NULL means no department

Types of JOIN

1. INNER JOIN

Returns only **matching records in both tables**

```
SELECT e.name, d.dept_name  
FROM employees e  
INNER JOIN departments d  
ON e.dept_id = d.dept_id;
```

Result:

- Only employees whose dept exists

Excludes:

- NULL dept_id
- Invalid dept_id (like 5)

2. LEFT JOIN (LEFT OUTER JOIN)

Returns:

- All records from **left table**
- Matching from right table
- Non-matching → NULL

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Result:

- All employees shown
- If no match → dept_name = NULL

3. RIGHT JOIN (Not Supported in SQLite)

SQLite does NOT support RIGHT JOIN directly

Alternative:

Swap tables + use LEFT JOIN

```
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e
ON e.dept_id = d.dept_id;
```

4. FULL JOIN (Not Supported in SQLite)

Not directly supported

Alternative:

Combine LEFT + RIGHT using UNION

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
```

UNION

```
SELECT e.name, d.dept_name
FROM departments d
LEFT JOIN employees e
ON e.dept_id = d.dept_id;
```

5. CROSS JOIN

Combines **every row with every row**

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Result:

- If 5 employees & 4 departments → 20 rows

Rarely used in real apps

Real-Life Understanding

JOIN Type	Meaning
INNER JOIN	Only matching data
LEFT JOIN	All from left + matching right
RIGHT JOIN	All from right + matching left
FULL JOIN	All data from both
CROSS JOIN	All combinations

Important Example (Most Asked)

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name IS NULL;
```

Meaning:

Employees who **do not have valid department**

Important Notes

- JOIN uses **ON** condition
- Always use **aliases (e, d)** for readability
- LEFT JOIN is most used in real apps

Practice Tasks

1. Show all employees with department name
2. Show employees with no department
3. Show all departments even if no employee
4. Count employees per department (GROUP BY + JOIN)

Subqueries (Nested Queries)

A **subquery** is a query inside another query.

Example 1: Subquery in WHERE

```
SELECT name, salary
FROM employees
```

```
WHERE salary > (  
    SELECT AVG(salary) FROM employees  
) ;
```

Meaning:

Employees earning **more than average salary**

Example 2: Subquery with IN

```
SELECT name  
FROM employees  
WHERE dept_id IN (  
    SELECT dept_id FROM departments WHERE dept_name = 'IT'  
) ;
```

Employees in IT department

Example 3: Subquery in FROM

```
SELECT dept_id, avg_salary  
FROM (  
    SELECT dept_id, AVG(salary) AS avg_salary  
    FROM employees  
    GROUP BY dept_id  
) ;
```

Creates temporary table

2. EXISTS

Checks if subquery returns any row

Example

```
SELECT name  
FROM employees e  
WHERE EXISTS (  
    SELECT 1  
    FROM departments d  
    WHERE d.dept_id = e.dept_id  
) ;
```

Employees whose department exists

NOT EXISTS

```
SELECT name  
FROM employees e  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM departments d  
    WHERE d.dept_id = e.dept_id  
) ;
```

Employees with **no valid department**

3. GROUP BY

Groups rows based on column

Example

```
SELECT dept_id, COUNT(*) AS total_employees  
FROM employees  
GROUP BY dept_id;
```

Count employees per department

4. HAVING

Used to filter grouped data
Works **after** GROUP BY

Example

```
SELECT dept_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY dept_id  
HAVING avg_salary > 40000;
```

Departments with avg salary > 40000

5. ORDER BY

Sorts result

Ascending (default)

```
SELECT * FROM employees  
ORDER BY salary;
```

Descending

```
SELECT * FROM employees  
ORDER BY salary DESC;
```

Multiple Columns

```
SELECT * FROM employees  
ORDER BY department ASC, salary DESC;
```

6. DISTINCT

Removes duplicate values

```
SELECT DISTINCT department FROM employees;
```

Unique departments

7. LIMIT + ORDER BY (Important Combo)

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Top 3 highest salaries

8. Complete Real-Life Query

```
SELECT d.dept_name, COUNT(e.emp_id) AS total, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING avg_salary > 30000
ORDER BY avg_salary DESC
LIMIT 5;
```

Meaning:

- Department name
- Total employees
- Average salary
- Only avg > 30000
- Sorted highest first
- Top 5 results

Practice Tasks

1. Find employees earning more than average salary
2. Count employees per department
3. Show departments with more than 2 employees
4. Show top 3 highest paid employees
5. Find employees whose department does not exist (EXISTS)

What are Window Functions?

Window functions perform calculations **across a set of rows (window)**
But **do NOT group rows** (unlike `GROUP BY`)

They return result **for each row**, not one row per group

Basic Syntax

```
FUNCTION_NAME(column)
OVER (
    PARTITION BY column
    ORDER BY column
)
```

Sample Table

```
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
```

```
    salary INTEGER  
);
```

1. ROW_NUMBER()

Assigns unique number to each row

```
SELECT name, department, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank_no  
FROM employees;
```

Highest salary → rank 1

2. PARTITION BY (Very Important)

Divides data into groups (like GROUP BY but keeps rows)

```
SELECT name, department, salary,  
ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS  
dept_rank  
FROM employees;
```

Ranking **within each department**

3. RANK()

Same rank for equal values, skips next rank

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;
```

Example:

- 50000 → rank 1
- 50000 → rank 1
- 40000 → rank 3

4. DENSE_RANK()

Same rank for equal values, **no skipping**

```
SELECT name, salary,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;
```

Example:

- 50000 → rank 1
- 50000 → rank 1
- 40000 → rank 2

5. NTILE()

Divides rows into groups

```
SELECT name, salary,
NTILE(3) OVER (ORDER BY salary DESC) AS group_no
FROM employees;
```

Divides into 3 groups

6. SUM() as Window Function

Running total

```
SELECT name, salary,
SUM(salary) OVER (ORDER BY salary) AS running_total
FROM employees;
```

Cumulative salary

7. AVG() as Window Function

```
SELECT name, department, salary,
AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Average salary per department (shown in each row)

8. LAG()

Get previous row value

```
SELECT name, salary,
LAG(salary) OVER (ORDER BY salary) AS prev_salary
FROM employees;
```

Compare with previous salary

9. LEAD()

Get next row value

```
SELECT name, salary,
LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

Real-Life Example

```
SELECT name, department, salary,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank
FROM employees;
```

Find top employee in each department

Window vs GROUP BY

Feature	GROUP BY	Window Function
Rows reduced	Yes	No

Feature	GROUP BY	Window Function
Result rows	One per group	Same rows
Use case	Summary	Analysis

Important Notes

- Always use `OVER ()`
- `PARTITION BY` = grouping
- `ORDER BY` = sorting inside window
- Works in SQLite (version 3.25+)

Practice Tasks

1. Rank employees by salary
2. Rank employees within department
3. Find running total of salary
4. Compare salary with previous employee
5. Divide employees into 4 groups

ACID Properties (Very Important)

1. Atomicity

All or nothing

If one query fails → rollback everything

2. Consistency

Database remains valid

Rules & constraints always satisfied

3. Isolation

Transactions don't affect each other

Safe for multiple users

4. Durability

Once committed → data is permanent

Transaction Commands in SQLite

1. BEGIN TRANSACTION

Start transaction

```
BEGIN TRANSACTION;
```

2. COMMIT

Save changes permanently

```
COMMIT;
```

3. ROLLBACK

Undo all changes

```
ROLLBACK;
```

Full Example (Bank Transfer)

```
BEGIN TRANSACTION;
```

```
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;
```

```
COMMIT;
```

If everything works → changes saved

Example with Error → Use ROLLBACK

```
BEGIN TRANSACTION;
```

```
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;
```

```
-- Suppose error happens here
```

```
ROLLBACK;
```

All changes cancelled

SAVEPOINT (Advanced)

Used inside transaction

Create Savepoint

```
SAVEPOINT sp1;
```

Rollback to Savepoint

```
ROLLBACK TO sp1;
```

Release Savepoint

```
RELEASE sp1;
```

Example with SAVEPOINT

```
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

SAVEPOINT sp1;

UPDATE accounts SET balance = balance + 100 WHERE id = 2;

-- If error happens
ROLLBACK TO sp1;

COMMIT;
```

Only second update cancelled

Auto-Commit in SQLite

By default:

- Every query runs in its own transaction
- Automatically committed

Use `BEGIN` when you want control

Important Notes

- Use transactions for:
 - Payments
 - Multiple related updates
 - Data integrity
- Always use **COMMIT** or **ROLLBACK**
- Without COMMIT → changes not saved

Practice Tasks

1. Create table `accounts` with balance
2. Insert 2 accounts
3. Perform money transfer using transaction
4. Try rollback manually
5. Use `SAVEPOINT`